SQLAlchemy

Complete Developer Handbook

ORM · Core · Async · Migrations · Best Practices

Python 3.10+	SQLAlchemy 2.x	Alembic	Async

Table of Contents

Section 1

Overview & Architecture

SQLAlchemy is Python's most powerful SQL toolkit and Object Relational Mapper (ORM). It provides two distinct but complementary layers of database interaction:

Layer	Module	Description
Core	sqlalchemy.sql	SQL expression language — close to raw SQL, maximum control
ORM	sqlalchemy.orm	High-level object mapping — work with Python classes & objects

■ **Which layer to use?** Use the ORM for everyday CRUD and relationships. Drop down to Core (or raw SQL) for complex reporting queries, bulk operations, or anything where fine-grained SQL control matters.

Section 2

Installation & Engine Setup

Installation

```
pip install sqlalchemy

pip install sqlalchemy[asyncio] # async support

pip install psycopg2-binary # PostgreSQL driver

pip install pymysql # MySQL driver

pip install asyncpg # async PostgreSQL driver
```

Connection Strings

```
from sqlalchemy import create_engine


# SQLite (file)
```

```
engine = create_engine("sqlite:///mydb.db")


# SQLite (in-memory)

engine = create_engine("sqlite:///:memory:")


# PostgreSQL

engine = create_engine("postgresql+psycopg2://user:password@localhost:5432/mydb")


# MySQL

engine = create_engine("mysql+pymysql://user:password@localhost/mydb")


# MS SQL Server

engine = create_engine("mssql+pyodbc://user:password@dsn")
```

Pool Configuration

```
engine = create_engine(

"postgresql://user:pass@localhost/db",

pool_size=10, # connections in pool

max_overflow=20, # extra connections allowed

pool_timeout=30, # seconds to wait

echo=True # log all SQL

)
```

Section 3

Declarative ORM Models

Base Setup

```
from sqlalchemy.orm import DeclarativeBase


class Base(DeclarativeBase):

pass
```

Defining Models

```
from sqlalchemy.orm import Mapped, mapped_column, relationship

from sqlalchemy import String, Integer, Boolean, DateTime, ForeignKey

from typing import Optional, List

from datetime import datetime


class User(Base):

__tablename__ = "users"


id: Mapped[int] = mapped_column(Integer, primary_key=True)

name: Mapped[str] = mapped_column(String(100), nullable=False)

email: Mapped[str] = mapped_column(String(200), unique=True)

age: Mapped[Optional[int]] = mapped_column(Integer, nullable=True)

is_active: Mapped[bool] = mapped_column(Boolean, default=True)
```

```
created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)


posts: Mapped[List["Post"]] = relationship("Post", back_populates="author")


class Post(Base):

__tablename__ = "posts"


id: Mapped[int] = mapped_column(Integer, primary_key=True)

title: Mapped[str] = mapped_column(String(200))

user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))

author: Mapped["User"] = relationship("User", back_populates="posts")
```

Create / Drop Tables

```
Base.metadata.create_all(engine) # create all tables

Base.metadata.drop_all(engine) # drop all tables
```

Section 4

Sessions & CRUD Operations

The Session is the unit of work — it tracks all ORM objects and coordinates writes to the database. Always use it as a context manager.

```
from sqlalchemy.orm import Session, sessionmaker


# Simple context manager

with Session(engine) as session:

...


# Factory (recommended for apps)

SessionLocal = sessionmaker(bind=engine, autocommit=False, autoflush=False)

with SessionLocal() as session:

...
```

Create

```
with Session(engine) as session:

user = User(name="Alice", email="alice@example.com", age=30)

session.add(user)

session.add_all([

User(name="Bob", email="bob@example.com"),

User(name="Carol", email="carol@example.com"),

])
```

```
session.commit()

session.refresh(user) # reload auto-generated fields

print(user.id)
```

Read / Query

```
from sqlalchemy import select, or_


with Session(engine) as session:

# By primary key

user = session.get(User, 1)


# All rows

users = session.scalars(select(User)).all()


# Filter (AND)

stmt = select(User).where(User.age > 18, User.is_active == True)


# Filter (OR)

stmt = select(User).where(or_(User.name == "Alice", User.name == "Bob"))


# Order, limit, offset

stmt = select(User).order_by(User.name.asc()).limit(10).offset(20)


# Count

from sqlalchemy import func
```

```
count = session.scalar(select(func.count()).select_from(User))
```

Update

```
with Session(engine) as session:

# Fetch-then-modify

user = session.get(User, 1)

user.name = "Alice Smith"

session.commit()


# Bulk update

from sqlalchemy import update

stmt = update(User).where(User.is_active == False).values(name="Inactive")

session.execute(stmt)

session.commit()
```

Delete

```
with Session(engine) as session:

# Fetch-then-delete

user = session.get(User, 1)

session.delete(user)

session.commit()


# Bulk delete

from sqlalchemy import delete
```

```
stmt = delete(User).where(User.is_active == False)

session.execute(stmt)

session.commit()
```

Section 5

Relationships

One-to-Many

```
class User(Base):

__tablename__ = "users"

id: Mapped[int] = mapped_column(primary_key=True)

posts: Mapped[List["Post"]] = relationship(

"Post", back_populates="author", cascade="all, delete-orphan"

)


class Post(Base):

__tablename__ = "posts"

id: Mapped[int] = mapped_column(primary_key=True)

user_id: Mapped[int] = mapped_column(ForeignKey("users.id"))

author: Mapped["User"] = relationship("User", back_populates="posts")
```

Many-to-Many

```
from sqlalchemy import Table, Column


# Association table

post_tags = Table(

"post_tags", Base.metadata,
```

```
Column("post_id", ForeignKey("posts.id"), primary_key=True),

Column("tag_id", ForeignKey("tags.id"), primary_key=True),

)


class Post(Base):

__tablename__ = "posts"

id: Mapped[int] = mapped_column(primary_key=True)

tags: Mapped[List["Tag"]]= relationship("Tag", secondary=post_tags,

back_populates="posts")


class Tag(Base):

__tablename__ = "tags"

id: Mapped[int] = mapped_column(primary_key=True)

name: Mapped[str] = mapped_column(String(50))

posts: Mapped[List["Post"]]= relationship("Post", secondary=post_tags,

back_populates="tags")
```

Loading Strategies

Choosing the right loading strategy is critical for performance — the wrong one causes N+1 query problems.

Strategy	Function	Queries	Best For
Lazy (default)	lazyload()	N+1	Single object, access rarely
Select IN	selectinload()	2	Collections, recommended default
Joined	joinedload()	1 JOIN	Many-to-one / one-to-one
Subquery	subqueryload()	2	Legacy / large collections

```
from sqlalchemy.orm import selectinload, joinedload


# Recommended: avoids N+1

stmt = select(User).options(selectinload(User.posts))

users = session.scalars(stmt).all()


# Join load (single JOIN)

stmt = select(User).options(joinedload(User.posts))
```

Section 6

Advanced Querying

Joins

```
# Join via relationship

stmt = select(User, Post).join(User.posts)


# Explicit join

stmt = select(User).join(Post, Post.user_id == User.id)


# Outer join

stmt = select(User).outerjoin(Post)
```

Aggregations & Group By

```
from sqlalchemy import func


stmt = (

select(User.name, func.count(Post.id).label("post_count"))

.join(Post, isouter=True)

.group_by(User.id)

.having(func.count(Post.id) > 2)

.order_by(func.count(Post.id).desc())

)
```

```
for name, count in session.execute(stmt):

print(f"{name}: {count} posts")
```

Subqueries

```
# Scalar subquery

subq = select(func.avg(User.age)).scalar_subquery()

stmt = select(User).where(User.age > subq)


# EXISTS

from sqlalchemy import exists

stmt = select(User).where(

exists().where(Post.user_id == User.id)

)
```

Raw SQL

```
from sqlalchemy import text


with Session(engine) as session:

result = session.execute(

text("SELECT * FROM users WHERE age > :min_age"),

{"min_age": 18}

)

for row in result:

print(row)
```

Section 7

Migrations with Alembic

Alembic is the de-facto migration tool for SQLAlchemy. It generates versioned migration scripts and tracks which migrations have been applied.

Setup

```
pip install alembic

alembic init alembic # initialise

alembic revision --autogenerate -m "init" # generate migration

alembic upgrade head # apply all

alembic downgrade -1 # roll back one

alembic history # view history
```

alembic/env.py — Point to your models

```
from myapp.models import Base

target_metadata = Base.metadata
```

■■ Always review auto-generated migrations before applying them in production. Alembic cannot detect all schema changes (e.g. column renames).

Section 8

Async SQLAlchemy

```
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession

from sqlalchemy.orm import sessionmaker
```

```
async_engine = create_async_engine(
"postgresql+asyncpg://user:pass@localhost/db"
)

AsyncSessionLocal = sessionmaker(
async_engine, class_=AsyncSession, expire_on_commit=False
)

async def get_users():
async with AsyncSessionLocal() as session:
result = await session.execute(select(User))
return result.scalars().all()
```

■ Use **asyncpg** for PostgreSQL and **aiosqlite** for SQLite async drivers. Set **expire_on_commit=False** to avoid lazy-load errors after commit in async contexts.

Section 9

Column Types Reference

SQLAlchemy Type	Python Type	SQL Type	Notes
Integer	int	INT	Standard integer
BigInteger	int	BIGINT	Large integer
String(n)	str	VARCHAR(n)	Variable-length string
Text	str	TEXT	Unbounded text
Float	float	FLOAT	Floating-point
Numeric(p,s)	Decimal	NUMERIC	Fixed precision
Boolean	bool	BOOLEAN	True/False
DateTime	datetime	DATETIME	Date + time
Date	date	DATE	Date only
Time	time	TIME	Time only
JSON	dict/list	JSON	PostgreSQL native JSON
Enum	str/Enum	ENUM	Fixed value set
LargeBinary	bytes	BLOB	Binary data
UUID	uuid.UUID	UUID	PostgreSQL UUID type

Section 10

Best Practices & Indexing

■ Always use context managers

Use `with Session(engine) as session:` to ensure connections are returned to the pool and transactions are finalized.

■ Avoid N+1 queries

Use selectinload() or joinedload() when you know you'll access a relationship inside a loop. Never rely on lazy loading in collection iterations.

■ expire_on_commit=False for async/APIs

In FastAPI or async contexts, expired attributes trigger implicit I/O. Set expire_on_commit=False on your sessionmaker.

■ Index queried columns

Add index=True to columns used in WHERE / ORDER BY clauses. Use composite indexes for multi-column filters.

■ Use bulk operations for large datasets

Prefer execute(insert()), execute(update()), and execute(delete()) over adding/modifying objects one-by-one in large loops.

■ Never commit in a loop

Batch your changes and commit once. Each commit is a round-trip to the DB.

■ Use Alembic from day 1

Even on small projects, track schema changes from the start. Retrofitting migrations later is painful.

Indexing Examples

```
# Single-column index

email: Mapped[str] = mapped_column(String(200), index=True, unique=True)


# Composite index
```

```
from sqlalchemy import Index


class User(Base):

__tablename__ = "users"

__table_args__ = (

Index("ix_name_email", "name", "email"),

Index("ix_active_created", "is_active", "created_at"),

)
```

Section 11

Quick Reference Card

Concept	What it does
`create_engine(url)`	Create DB connection pool
`Base.metadata.create_all(e)`	Create tables from models
`Session(engine)`	Open a unit-of-work session
`session.add(obj)`	Stage a new object
`session.commit()`	Persist staged changes
`session.rollback()`	Undo uncommitted changes
`session.get(Model, pk)`	Fetch by primary key
`select(Model)`	Build a SELECT statement
`session.scalars(stmt).all()`	Execute & return ORM objects
`session.execute(stmt)`	Execute Core-style statement
`selectinload(rel)`	Eager-load a collection (2 queries)
`joinedload(rel)`	Eager-load via JOIN (1 query)
`func.count() / func.avg()`	SQL aggregate functions
`text('SELECT ...')`	Raw SQL with bind params
`alembic upgrade head`	Apply all pending migrations

■ **Further Reading:** Official docs at docs.sqlalchemy.org • Alembic docs at alembic.sqlalchemy.org • Async patterns in the SQLAlchemy async tutorial.